

WEB PORTAL

a simple multi-page interface with registration and task management

MUHAMMAD IBRAHIM SHOEB

BSc in Computer Science Engineering, BME
OZLVV3



github.com/ibrahimify/my-bootstrap-portal

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
INTRODUCTION.....	2
SYSTEM OVERVIEW.....	2
HOME PAGE.....	2
Screenshot.....	3
REGISTRATION PAGE.....	3
Screenshot.....	4
TO-DO PAGE.....	4
Screenshot.....	5
TECHNOLOGY USED.....	5
CONCLUSION.....	6

INTRODUCTION

This document provides an overview of the three-page web application created for the Mobile- and Web-based Software homework assignment. The project consists of a Home page, a Registration page, and a functional To-Do application.

The purpose of this report is to demonstrate that all assignment requirements were fulfilled, including navigation, responsiveness, form handling via GET parameters, and dynamic To-Do list functionality with persistence.

SYSTEM OVERVIEW

The web application consists of three linked pages:

1. **Home Page:** Introduction and navigation
2. **Registration Page:** User form with field validation
3. **To-Do Page:** JavaScript-powered to-do management with persistent storage

Navigation is consistent across all pages and adapts to mobile layout using Bootstrap's responsive navbar.

Custom styling is placed in an external `styles.css` file, ensuring clean and maintainable code. The JavaScript logic is contained in `todo.js`.

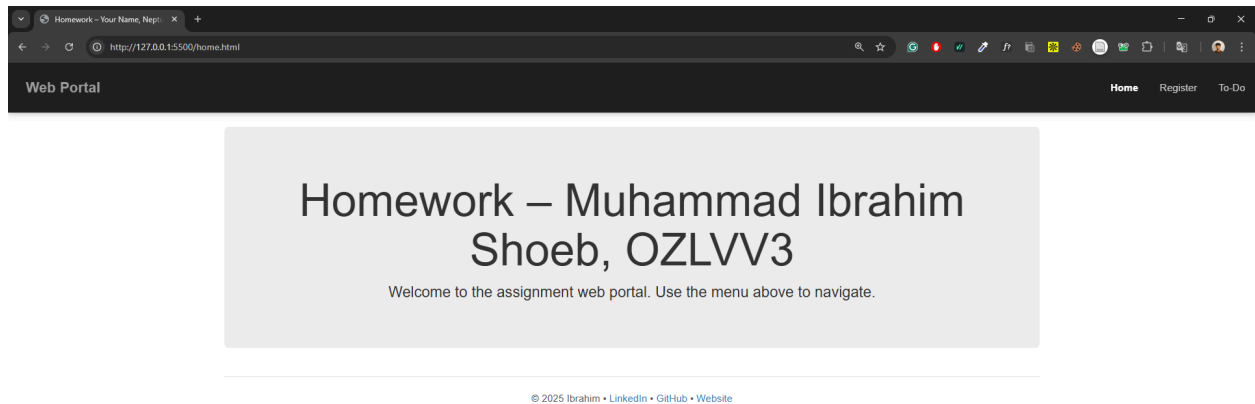
HOME PAGE

The Home page displays the assignment title: “**Homework – Muhammad Ibrahim Shoeb, OZLVV3**”

The layout includes:

- Navigation bar
- Jumbotron header
- Footer section

Screenshot



REGISTRATION PAGE

The Registration page is built using a centered Bootstrap panel. The form includes all required fields and client-side constraints:

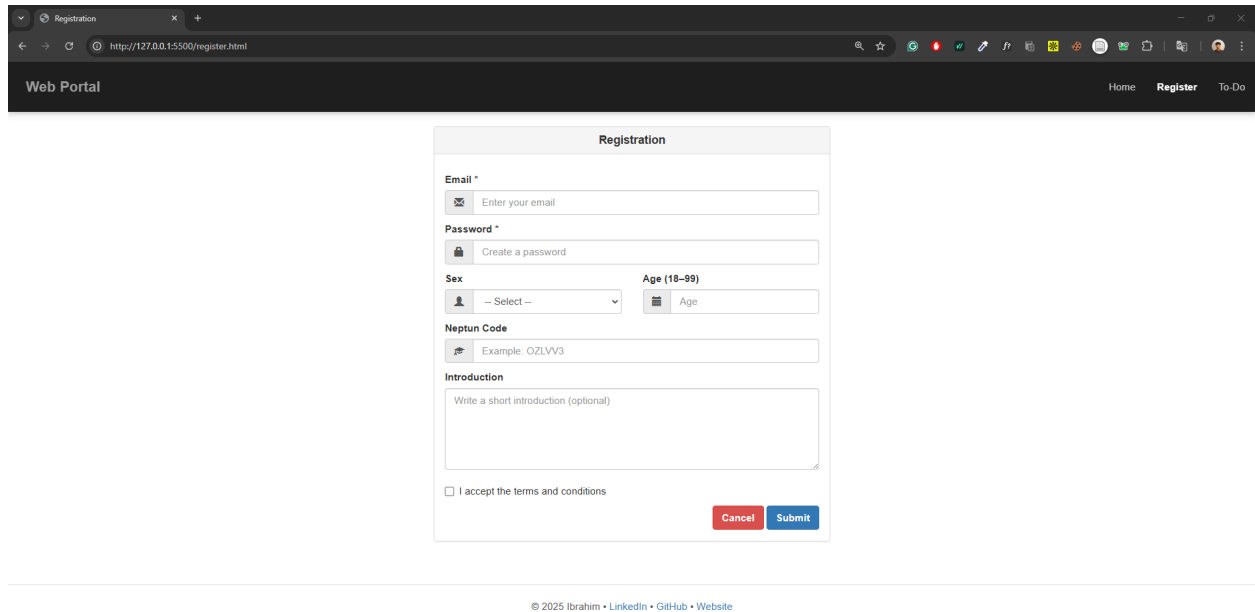
- Required input fields
- Dropdown menus
- Age validation using min/max attributes
- Neptun code validation using a regular expression pattern
- Introduction textarea (5 rows)
- A required checkbox for accepting terms

The Cancel button redirects to the Home page.

The Submit button sends data to the To-Do page using HTTP GET parameters exactly as the assignment specifies.

Processing the data on the destination page is intentionally not implemented, as stated in the task.

Screenshot



The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5500/register.html`. The browser's address bar also shows the page title "Registration". The browser's toolbar includes search, star, and various extension icons. The browser's navigation bar shows "Home", "Register", and "To-Do". The main content area displays a registration form titled "Registration". The form contains the following fields and controls:

- Email ***: A text input field with a placeholder "Enter your email".
- Password ***: A text input field with a placeholder "Create a password".
- Sex**: A dropdown menu with a placeholder "-- Select --".
- Age (18-99)**: A text input field with a placeholder "Age".
- Neptun Code**: A text input field with a placeholder "Example: OZLVV3".
- Introduction**: A text area with a placeholder "Write a short introduction (optional)".
- ☐ I accept the terms and conditions
- Cancel** and **Submit** buttons.

At the bottom of the page, there is a footer with the text: © 2025 Ibrahim • [LinkedIn](#) • [GitHub](#) • [Website](#)

TO-DO PAGE

The To-Do page contains a fully functional task management system created in vanilla JavaScript.

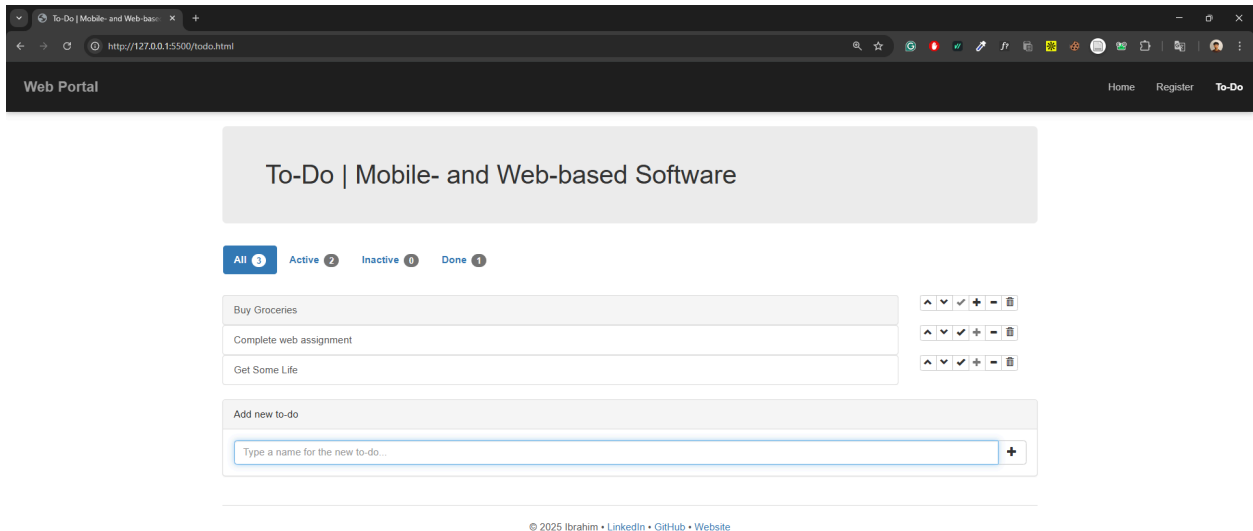
Implemented Features

- Add new tasks
- Change task state (active, inactive, done)
- Reorder tasks using Up/Down buttons
- Remove tasks
- Real-time filtering by tabs
- Badge counters dynamically updated
- Persistent storage using `localStorage`
- Buttons generated inside `renderTodos()` per requirement
- Works correctly even when lists are filtered

The application does not crash on invalid actions and handles edge cases such as empty

lists or boundary movement.

Screenshot



TECHNOLOGY USED

The project uses the following technologies:

- **HTML5** for structural layout
- **Bootstrap 3.3.7** for responsive grid and components
- **CSS (styles.css)** for custom styling (navbar, forms, footer, spacing)
- **JavaScript (todo.js)** for To-Do behavior and localStorage
- **No backend**, consistent with assignment requirements

CONCLUSION

This project provided practical experience in building a small yet complete client-side web application using HTML, CSS, Bootstrap, and JavaScript. Throughout the implementation, the key concepts of responsive design, structured page layout, form handling through GET requests, and dynamic DOM manipulation were applied. The To-Do module strengthened understanding of state management, event handling, and persistence using localStorage.

The application meets all requirements of the assignment and functions reliably across different screen sizes. While the current solution fulfills the scope, several enhancements could be explored in the future, such as adding server-side storage, user accounts, improved accessibility, or expanding the To-Do system with deadlines, categories, or reminders. These additions would transform the simple portal into a more robust and scalable application.

Acknowledgments: Special thanks to **Mohammad Majed Saleem** for academic support and mentorship.